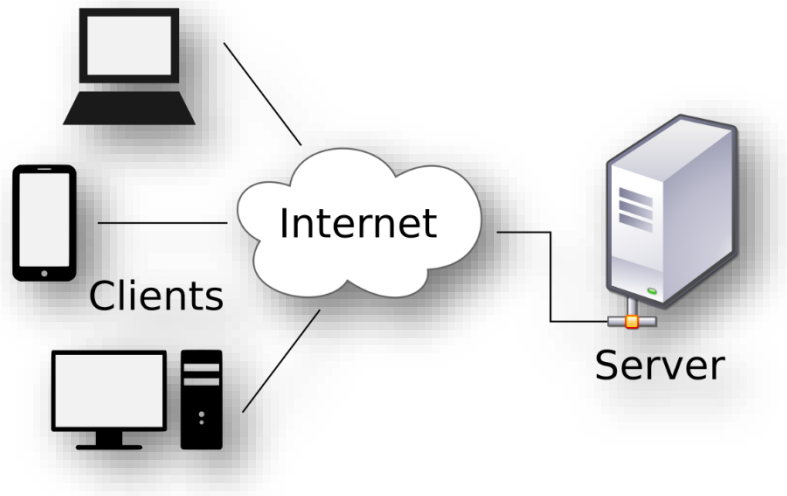
Høgskulen
på Vestlandet



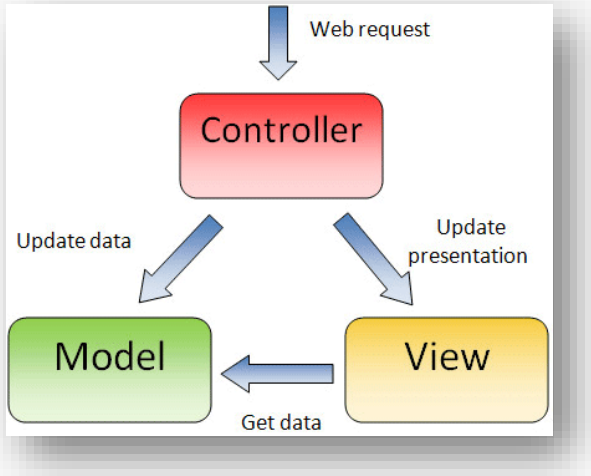
DAT108 - Programmering og webapplikasjoner

F15 Web Oversikt Intro

Lars-Petter Helland
Bergen
5. oktober 2026

Om modulen vi har kalt "Spring MVC"

- Starter i dag og går over 3 uker!
  - 6 forelesninger (F15-F20)
  - En del eksempler og frivillige øvingsoppgaver
  - Én oblig (Oblig4)
- Modulen handler om å skrive Java-programmer som skal kjøre på en webtjener (*web server*).



Bygger på ...

- I denne modulen bruker, og setter vi sammen, mange ulike teknologier som dere har lært om tidligere i studiet:
  - HTML og CSS (fra ING102)
  - Java (fra DAT100, DAT102 og DAT108)
  - ORM / JPA (fra DAT107)
  - JavaScript og JSON (fra DAT108)
- Vi antar altså at dere behersker disse til en viss grad !

Pensum

Som vanlig er **forelesninger**, **eksempler** og **øvingsoppgaver** pensum.

I tillegg, se Pensumoversikt i Canvas for **lesepensum.**

NB! Nytt i år er at vi bytter malspråk (template language) fra JavaServer Pages (JSP) til det mer moderne Thymeleaf, https://www.thymeleaf.org

(, men pensumlisten er ikke oppdatert helt ennå)



Læringsutbytte - Noen punkter fra emneplanen

- Grunnleggende om HTTP. Request-response. REST.
- Forklare kommunikasjonen som skjer mellom klient og tjener i en webapplikasjon
- Tjenersideprogrammering med Spring MVC og JSP/Thymeleaf el.l.
- Skrive programkode for generering av dynamiske websider
- Grunnleggende om logisk arkitektur i webapplikasjoner. MVC. Lagdeling.
- Utforming av web-baserte interaktive brukergrensesnitt
- Enkel tilgangskontroll og brukerhåndtering

Vi starter med noen grunnleggende begreper



Internett* (*the Internet*) og IP-adresser

* https://sprakradet.no/spraksporsmal-og-svar/internett-eller-internett/

Jeg spør GPT UiO "**Hva er internett?**", og får som svar:

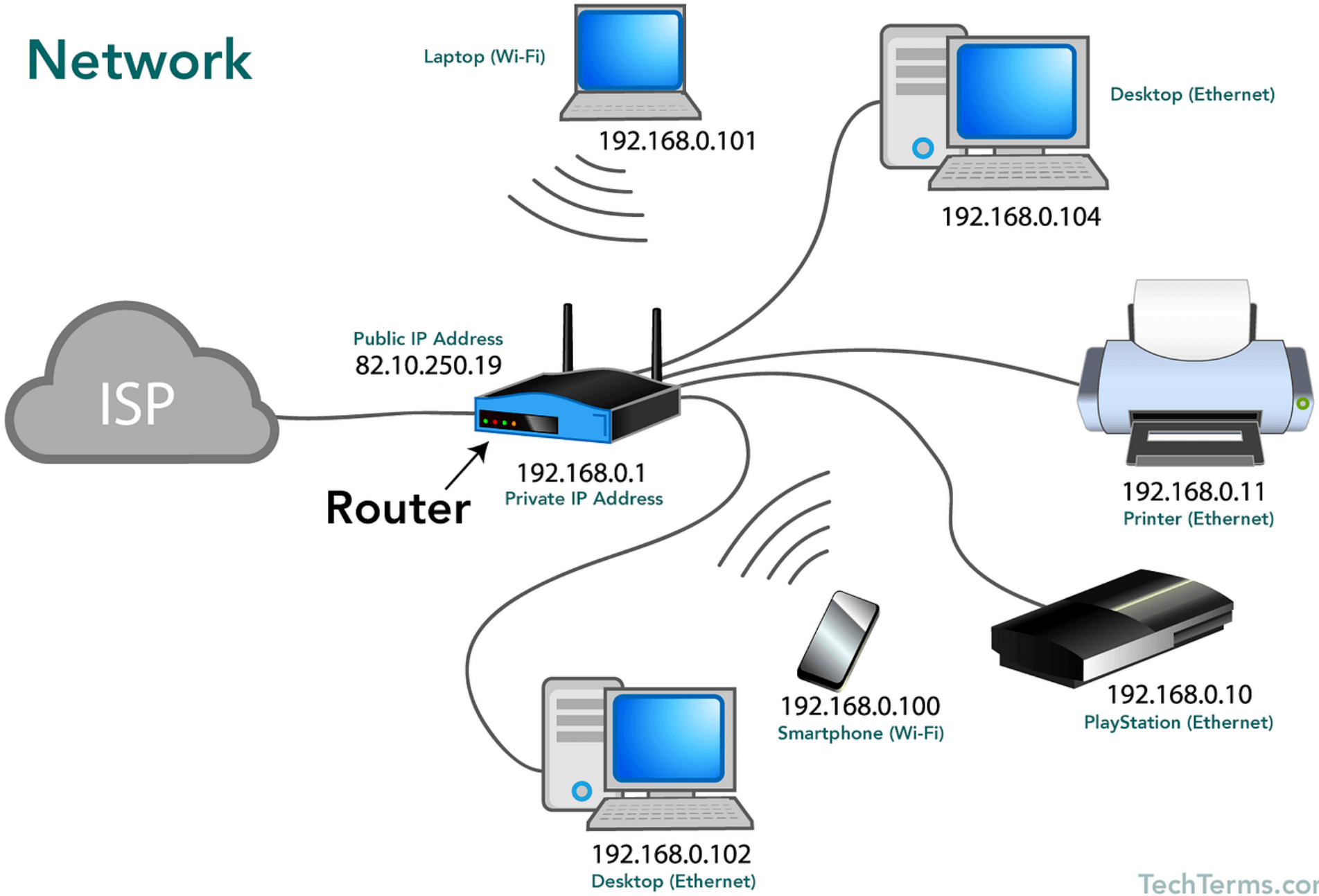
> *"Internett er et globalt nettverk av datanettverk som kobler millioner av datamaskiner sammen over hele verden. Det gjør det mulig for maskiner å kommunisere med hverandre og utveksle informasjon på mange forskjellige måter, inkludert tekst, bilder, video og lyd. ..."*

Jeg spør GPT UiO "**Hva er en IP-adresse?**", og får som svar:

> *"En IP-adresse (Internet Protocol-adresse) er en unik tallrekke som brukes til å identifisere en enhet på et nettverk ..."*

- Alle maskiner som er koblet til internett har en IP-adresse som identifiserer denne maskinen.
- Tenk på det som tilsvarende til at alle mobiltelefoner har et unikt telefonnummer.
- Vi bruker IP-adressen til å kontakte og "snakke med" en maskin.

Network



TechTerms.com

IP-adresser, DNS, domenenavn

- IP-adressen til VG sin web-tjener er 195.88.55.16
- Jeg kan (i teorien) skrive https://195.88.55.16 i nettleseren for å komme til VG, men sikkerhets- og andre innstillinger tillater sannsynligvis ikke dette.
- Men på samme måte som vi har en adressebok på telefonen for å slippe å huske alle telefonnumre, så har internett en adressebok for at vi skal slippe å huske alle IP-adressene.

Jeg spør GPT UiO "**Hva er DNS?**", og får som svar:

> *"DNS (Domain Name System) er en ... desentralisert navngivningstjeneste for datamaskiner, tjenester eller andre ressurser som er koblet til internett ... . Hovedformålet med DNS er å oversette menneskevennlige **domenenavn** (som www.example.com) til IP-adresser (som 192.0.2.1), som datamaskiner bruker for å identifisere hverandre på internett.*
>
> *Uten DNS ville brukere måtte huske og skrive inn IP-adressene til nettstedene de ønsket å besøke, noe som ville være upraktisk og vanskelig."*

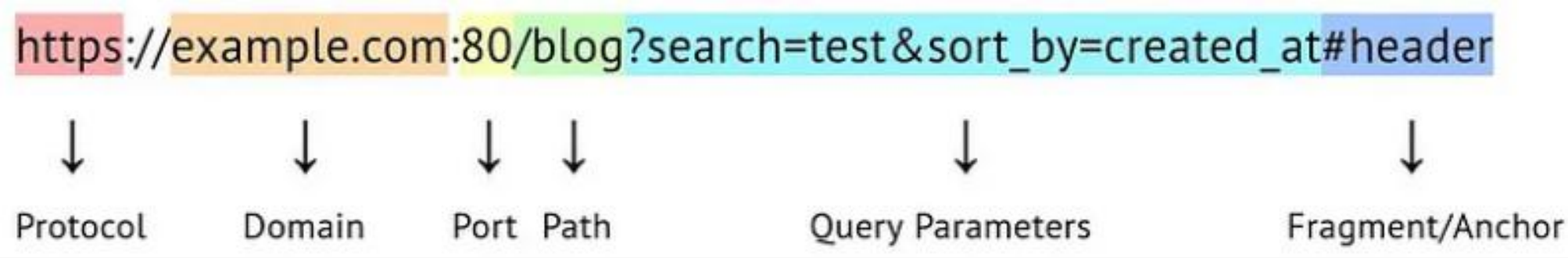
Domenenavn og URL-er

- Navngivingstjenesten DNS mapper altså domenenavn til IP-adresser, slik at vi f.eks. kan skrive http://vg.no i stedet for http://195.88.55.16 *(Her har altså VG bedt om (og betalt for) å "eie" og få en oppføring i DNS-adresseboken for domenenavnet **vg.no**)*

Jeg spør GPT UiO "**Hva er en URL?**", og får som svar:

"En URL (Uniform Resource Locator) er en standardisert adresse som brukes for å spesifisere ... en ressurs på internett. ...

*En typisk URL består av følgende: **Protokoll**, **Domene**, **Port**, **Sti**, **Spørrestreng**, **Fragment**. ..."*



Protokollen HTTP

Jeg spør GPT UiO "**Hva er HTTP?**", og får som svar:

> *"HTTP (Hypertext Transfer Protocol) er en standard protokoll som brukes for å overføre hypertekst (som webdokumenter) over internett. HTTP er grunnlaget for datakommunikasjon på **Verdensveven** (**World Wide Web**) og gjør det mulig for nettlesere og webservere å kommunisere med hverandre. ...*
>
> *HTTP fungerer etter en klient-server-modell:*
>
> - ***Klient**: Typisk en nettleser (som Chrome, Firefox, Safari), som sender en forespørsel (request) til en webserver.*
> - ***Server**: En webserver (som Apache, Nginx) som mottar forespørselen, behandler den og sender et svar (response) tilbake til klienten."*

World Wide Web

Jeg spør GPT UiO "**Hva er WWW?**", og får som svar:

> *"WWW, eller World Wide Web, er et informasjonssystem på internett som gjør det mulig for dokumenter og andre ressurser å kobles sammen ved hjelp av hypertekst-lenker. World Wide Web ble oppfunnet av den britiske forskeren Tim Berners-Lee i 1989 mens han arbeidet ved CERN (European Organization for Nuclear Research). ...*
>
> *WWW består av flere nøkkelkomponenter:* ***Hypertekst****,* ***HTML****,* ***HTTP****,* ***Nettleser****,* ***Webserver***
>
> *For å få tilgang til WWW, gjør vi følgende: Åpne en nettleser -> Skriv inn en URL -> Nettleser sender en HTTP-forespørsel -> Webserver sender tilbake en HTTP-respons -> Nettleser viser websiden.*

Legg merke til at **www (webben) og HTTP er to sider av samme sak**!

Når vi snakker om webben, snakker vi altså om **HTTP, nettlesere og webtjenere** !

(PS! Det er mange andre protokoller på internett ,f.eks. for epost, filoverføring, telefoni, direktekommunikasjon, osv..., men dette er ikke web !)

Oppsummering begreper

- Internett - *Et globalt nettverk som kobler millioner av datamaskiner sammen*
- IP-adresse - *En unik tallrekke som brukes til å identifisere en enhet på et nettverk*
- Domenenavn - *Et menneskevennlig navn som er knyttet til en IP-adresse via DNS*
- URL - *En standardisert streng som brukes for å spesifisere en ressurs på internett*
- HTTP - *En standard protokoll som brukes for å overføre hypertekst* over internett*
- (Web)Klient - *Typisk en nettleser (som Chrome, Safari), som sender HTTP-forespørsler*
- (Web)Server - *En applikasjon som mottar og behandler forespørsler, og sender svar tilbake*
- Web - *Et informasjonssystem basert på HTTP, og med følgende nøkkelkomponenter: Hypertekst, HTML, HTTP, Nettleser, Webserver*

*mange ulike typer data kan overføres med HTTP, ikke bare hypertekst

Det er altså dette som er omgivelsene for våre Java Web-applikasjoner !!!

Gangen i en forespørsel til en dynamisk webside

- Vi skal nå se et eksempel på en web-applikasjon, og hva som er "gangen" i det fra vi skriver URL-en i nettleseren til vi får et svar.

Vårt første webapp-eksempel

Prosjekt: 📂 f15-hilsen-personliste

Vi skal se på 6 ulike eksempler



Hilsen:

1. Webtjeneren returnerer ren tekst, `/teksthilsen`
   Full URL: http://localhost:8080/f15app/teksthilsen?navn=Per
2. Webtjeneren returnerer JSON, `/jsonhilsen`
3. Webtjeneren returnerer ferdig HTML-side (SSR), `/htmlhilsen`

Personliste:

4. Webtjeneren returnerer ferdig HTML-side (SSR), `/personliste-ssr`
5. Webtjeneren returnerer JSON, `/api/personer`
6. Webtjeneren returnerer HTML-mal og JavaScript (CSR), `/personliste-csr.html`

Whiteboard-tegninger fra timen

- Webtjeneren returnerer ren tekst, `/teksthilsen`

Whiteboard-tegninger fra timen

- Webtjeneren returnerer JSON, `/jsonhilsen`

Model-View-Controller for server-generert HTML

- Webtjeneren returnerer ferdig HTML-side (SSR), `/htmlhilsen`

Model-View-Controller for server-generert HTML

- Webtjeneren returnerer ferdig HTML-side (SSR), `/personliste-ssr`

Whiteboard-tegninger fra timen

- Webtjeneren returnerer personlisten som JSON, `/api/personer`

Klient-generert HTML, dvs. JavaScript bygger HTMLen i nettleseren.

- Webtjeneren gir tom HTML, JavaScript, og JSON-data (CSR), `/personliste-csr.html`

Ulike tilnærminger til generering av nettsider

Jeg spør GPT UiO "**Hva er forskjellen på client side rendering og server side rendering?**", og får som svar:

"Server-side rendering (SSR) og Client-side rendering (CSR) er to forskjellige tilnærminger til hvordan innhold på websider genereres og presenteres til brukeren. Begge metodene har sine fordeler og ulemper, og valget mellom dem kan avhenge av spesifikke krav og kontekster for et prosjekt. ...

- ***Server-Side Rendering*** *(SSR): Serveren genererer HTML og sender det ferdige dokumentet til klienten. Bedre for SEO og raskere første visning, men kan føre til økt serverbelastning og potensiell forsinkelse ved brukerhandlinger. ..."*
- ***Client-Side Rendering*** *(CSR): JavaScript genererer HTML på klienten. Bedre for interaktivitet og lastegang, men kan ha dårligere søkemotoroptimalisering og lengre initiell innlasting.*

Spring MVC

I DAT108 skal vi starte med Server Side Rendering, men ..

NYTT i høst er at vi også skal se litt på REST, API-er og enkel Client Side Rendering med JavaScript.

Vi skal bruke Java-rammeverket **Spring MVC** (i Spring Boot).

Jeg spør Google "**Hva er Spring MVC?**", og får som svar:

> *"Spring MVC er et rammeverk i Spring-økosystemet som brukes til å bygge web-applikasjoner og REST-API-er i Java.*
>
> *forts. neste side ...*

Model-View-Controller og Spring MVC

Det er bygget på Servlet-API-et og følger det klassiske Model-View-Controller designmønsteret:

- ***M**odel: Dataene og forretningslogikken (f.eks. en bruker eller et produkt).*
- ***V**iew: Det brukeren ser (HTML, eller JSON/XML-responser i moderne API-er).*
- ***C**ontroller: Koden som tar imot HTTP-forespørsler (@RestController / @Controller), snakker med databasen, og sender svaret tilbake til brukeren.*

Videre

Fredag:

- Se litt mer på hvordan en Spring Boot- / Spring MVC-applikasjon er bygget opp og fungerer.
- Se på utviklingsprosessen. Hvordan går vi frem for å utvikle, teste og debugge en webapp.

Hva dere bør gjøre i uken:

- Få eksempelet fra denne timen til å kjøre på egen PC!